Projet Reinforcement Learning

Conduite autonome sur Highway-Env

Amine M'ZALI · Mehdi SAADI · Samy Bouaissa

BDML 2 – Efrei Paris

Cours : Reinforcement Learning (Victor Morand)

25 mars 2026

Table des matières

1 Introduction **2**
1.1 Objectif . . . 2
1.2 L'environnement . . . 2

2 Algorithmes utilisés **2**
2.1 DQN via Stable-Baselines3 (value-based, off-policy) . . . 2
2.2 PPO via Stable-Baselines3 (policy gradient, on-policy) . . . 3
2.3 DQN implémenté à la main (PyTorch) . . . 3

3 Exploration des hyperparamètres **3**
3.1 Grid search (Phase 1) . . . 3
3.2 Impact de gamma . . . 4
3.3 Impact du learning rate . . . 4

4 Résultats et évolution par phases **4**
4.1 Problèmes identifiés en Phase 2 (100k steps) . . . 4
4.2 Corrections appliquées (Phases 3 et 4) . . . 5
4.3 Résultats finaux . . . 5

5 Analyse comparative **6**
5.1 Benchmark final . . . 6
5.2 Off-policy vs On-policy . . . 6
5.3 SB3 vs implémentation manuelle . . . 6
5.4 Courbe d'entraînement du DQN manual . . . 7

6 Overtraining et variance en RL **7**
6.1 Le paradoxe de l'overtraining . . . 7
6.2 Variance inter-runs . . . 7
6.3 L'approche itérative comme clé du succès . . . 7

7 Bonus : Racetrack **8**

8 Conclusion **8**

A Structure du code **10**

1 Introduction

1.1 Objectif

L'objectif de ce projet est d'entraîner un agent de Reinforcement Learning à conduire sur une autoroute simulée dans l'environnement `highway-v0` de la librairie highway-env. L'agent doit apprendre à :

- Rouler le plus vite possible,
- Éviter les collisions avec les autres véhicules,
- Privilégier la voie de droite.

Étant un groupe de 3, nous avons implémenté **3 algorithmes** au lieu de 2, dont **un implémenté entièrement à la main** (sans Stable-Baselines3).

1.2 L'environnement

L'environnement simule une autoroute à 3 voies avec 40 véhicules agressifs (`AggressiveVehicle`). Le véhicule ego dispose de 5 actions discrètes :

Index	Action	Description
0	`LANE_LEFT`	Changer de voie vers la gauche
1	`IDLE`	Maintenir la trajectoire actuelle
2	`LANE_RIGHT`	Changer de voie vers la droite
3	`FASTER`	Accélérer
4	`SLOWER`	Ralentir

Table 1 – Espace d'actions (DiscreteMetaAction).

L'observation est une matrice 5×5 (5 véhicules proches, 5 features : `presence`, x, y, v_x, v_y), normalisée dans $[-1, 1]$.

La récompense combine vitesse (bonus linéaire entre 20–30 m/s), collision (pénalité -1, fin d'épisode) et voie de droite (bonus 0.1), normalisée dans $[0, 1]$.

La configuration d'évaluation est volontairement difficile : espacement initial très faible (0.1), conducteurs agressifs, durée de 40 secondes.

2 Algorithmes utilisés

2.1 DQN via Stable-Baselines3 (value-based, off-policy)

Deep Q-Network [1] approxime la fonction Q optimale $Q^*(s, a)$ avec un réseau de neurones. Deux innovations stabilisent l'entraînement :

- **Experience Replay Buffer** : les transitions $(s, a, r, s', \text{done})$ sont stockées et échantillonnées aléatoirement, brisant la corrélation temporelle.
- **Target Network** : une copie du Q-network, mise à jour tous les N steps, fournit des cibles stables :

$$y = r + \gamma \cdot \max_{a'} Q_{\text{target}}(s', a')$$

La loss est l'erreur TD quadratique :

$$\mathcal{L} = \mathbb{E}\left[(Q(s, a) - y)^2\right]$$

Hyperparamètres retenus (après grid search) : $\gamma = 0.9$, lr schedule $10^{-3} \to 10^{-4}$ (décroissance linéaire), `target_update_interval`= 100, buffer= 15 000, batch= 32, réseau MLP $[256, 256]$.

2.2 PPO via Stable-Baselines3 (policy gradient, on-policy)

Proximal Policy Optimization [2] est un algorithme actor-critic. L'acteur optimise directement la politique $\pi_\theta(a|s)$, tandis que le critique estime $V(s)$ pour calculer l'avantage :

$$A_t = R_t - V(s_t)$$

PPO contraint les mises à jour via le clipping du ratio d'importance-sampling :

$$\mathcal{L}^{\text{CLIP}} = \mathbb{E}\left[\min\left(r_t(\theta)A_t,\ \text{clip}\left(r_t(\theta),\ 1-\varepsilon,\ 1+\varepsilon\right)A_t\right)\right]$$

où $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$.

Contrairement au DQN, PPO est **on-policy** : il collecte des données avec la politique courante, met à jour, puis les jette. Moins sample-efficient mais plus stable.

Hyperparamètres retenus : $\gamma = 0.9$, lr schedule $3\times10^{-4} \to 3\times10^{-5}$, `clip_range`= 0.2, `n_steps`= 512, `n_epochs`= 10, `ent_coef`= 0.01, réseau MLP $[256, 256]$.

2.3 DQN implémenté à la main (PyTorch)

Pour démontrer notre compréhension du DQN, nous l'avons implémenté entièrement en PyTorch :

- **QNetwork** : MLP à 2 couches cachées de 256 neurones + ReLU. Input = observation aplatie (25 dim), output = 5 Q-values.
- **ReplayBuffer** : buffer circulaire de capacité 15 000.
- **Target Network** : copie dure tous les 50 gradient steps.
- **Epsilon-greedy** : décroissance linéaire de $\varepsilon = 1.0$ à $\varepsilon = 0.05$ sur 5 000 gradient steps ($\sim$350 épisodes).

La méthode `predict()` est rendue compatible avec l'interface SB3 pour que les fonctions `evaluate()` et `record_video()` fonctionnent directement.

Différence clé avec SB3 : pas d'environnements vectorisés pendant l'entraînement, pas de normalisation automatique, contrôle total de la boucle d'entraînement.

3 Exploration des hyperparamètres

3.1 Grid search (Phase 1)

Nous avons testé 15 configurations (5 par algorithme) avec un budget réduit (30k steps SB3, 200 épisodes DQN manual), puis évalué sur `EVAL_CONFIG` (15 épisodes).

Algo	$\gamma = 0.8$	$\gamma = 0.9$	$\gamma = 0.99$
DQN SB3	13.79	**28.81**	26.13
PPO SB3	23.24	**26.08**	21.19
DQN Manual	**28.46**	26.95	21.14

Table 2 – Meilleure reward par γ (meilleur lr pour chaque config).

3.2 Impact de gamma

$\gamma = 0.9$ est le sweet spot pour les algorithmes SB3 : il offre un horizon de planification d'environ 10 steps ($1/(1-\gamma)$), suffisant pour anticiper les véhicules proches sans accumuler trop d'incertitude.

Le DQN manual préfère $\gamma = 0.8$ (horizon ∼5 steps) car notre implémentation plus simple (pas d'envs vectorisés, gradient steps par transition) bénéficie d'un horizon plus court qui stabilise les cibles Q.

3.3 Impact du learning rate

- **DQN SB3** a besoin d'un lr élevé (10^{-3}) pour converger rapidement à 30k steps.
- **PPO** préfère un lr plus conservatif (3×10^{-4}) : le clipping protège déjà contre les mises à jour trop agressives.
- **DQN manual** fonctionne mieux avec un lr modéré (5×10^{-4}) offrant le meilleur compromis convergence/stabilité ($\sigma = 1.2$, remarquablement faible).

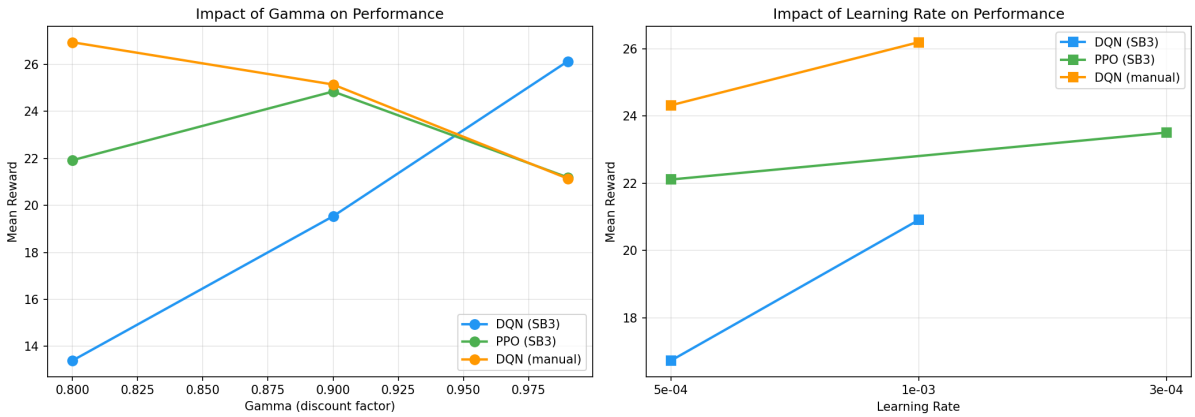


FIGURE 1 – Impact de γ (gauche) et du learning rate (droite) sur la performance.

4 Résultats et évolution par phases

4.1 Problèmes identifiés en Phase 2 (100k steps)

L'entraînement à pleine puissance a révélé des problèmes majeurs :

Algo	Phase 2 final	Phase 2 best	Problème identifié
DQN SB3	12.92	23.98 (70k)	Q-value overestimation
PPO SB3	23.53	28.63 (35k)	Distribution shift on-policy
DQN Manual	19.73	N/A	Epsilon decay trop lent

TABLE 3 – Résultats Phase 2 et problèmes diagnostiqués.

DQN SB3 – Q-value overestimation. Avec lr $= 10^{-3}$ fixe, les Q-values sont surestimées à cause de l'opérateur max dans la cible $y = r + \gamma \cdot \max_{a'} Q_{\text{target}}(s', a')$. Le modèle passe de 23.98 (70k) à 12.92 (100k), soit −46%.

PPO SB3 – Distribution shift. Étant on-policy, quand la politique s'améliore, la distribution d'états change. Le clipping limite la chute (-32% vs -46% pour DQN) et permet un rebond partiel.

DQN Manual – Bug epsilon decay. Configuré en gradient steps (15 000), l'agent avait encore 62% d'actions aléatoires à l'épisode 400. L'early stop se déclenchait car la reward d'entraînement (polluée par l'exploration) ne progressait plus, alors que la politique sous-jacente était potentiellement bonne.

4.2 Corrections appliquées (Phases 3 et 4)

Algo	Correction	Impact
DQN SB3	lr scheduler $10^{-3} \to 10^{-4}$	Élimine l'effondrement post-pic
DQN SB3	`target_update` : 50 → 100	Ralentit la propagation des erreurs
PPO SB3	lr scheduler $3\times10^{-4} \to 3\times10^{-5}$	Stabilise le training tardif
PPO SB3	`n_steps` : 256 → 512, `ent_coef`= 0.01	Gradients plus stables
PPO SB3	Budget 80k + copie auto du best	Évite l'overtraining
DQN Manual	`epsilon_decay` : 15 000 → 5 000	**Fix critique** : $\varepsilon = 0.05$ dès ép. 275
DQN Manual	Eval déterministe périodique	Best model basé sur la vraie perf.
DQN Manual	Gradient clipping (`max_norm`= 10)	Stabilise l'entraînement tardif
DQN Manual	1 000 épisodes sans early stop	Exploitation complète

Table 4 – Corrections ciblées par algorithme.

4.3 Résultats finaux

Algo	Phase 1	Phase 2	Phase 3	Phase 4
DQN SB3	28.81	12.92	**27.14**	27.14
PPO SB3	26.08	23.53	21.92	**27.88**
DQN Manual	28.46	19.73	8.35	**27.54**

Table 5 – Évolution des rewards moyennes à travers les 4 phases expérimentales.

Les 3 algorithmes convergent vers ∼27–28 de reward en Phase 4. Malgré des architectures fondamentalement différentes, les 3 approches atteignent des performances similaires une fois correctement diagnostiquées et corrigées.

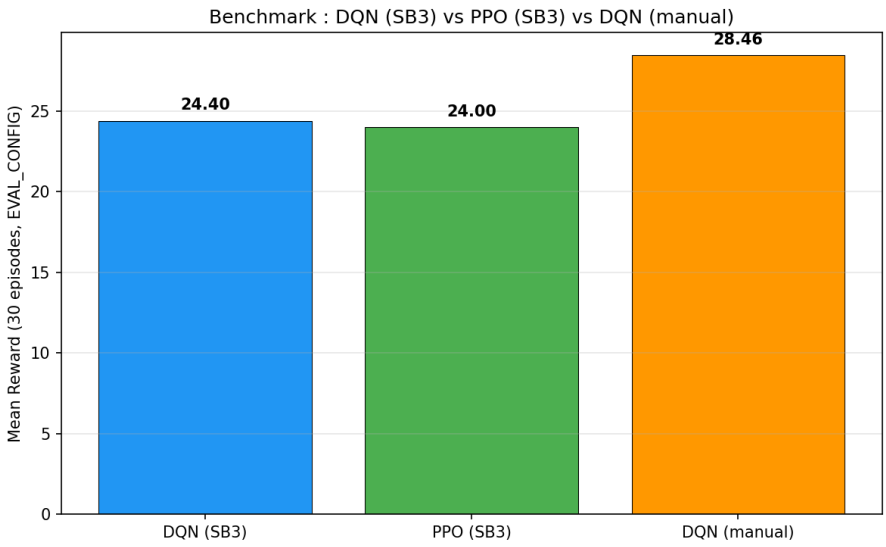


FIGURE 2 – Comparaison finale des 3 algorithmes (30 épisodes, `EVAL_CONFIG`).

5 Analyse comparative

5.1 Benchmark final

Algo	Reward	Std	Ep. Length	Survie
PPO SB3	**27.88**	**4.80**	**38.2**	95.5%
DQN Manual	27.54	7.35	36.6	91.5%
DQN SB3	27.14	8.17	34.8	87.0%

TABLE 6 – Benchmark final (30 épisodes, `EVAL_CONFIG`).

PPO est le meilleur modèle global : meilleure reward, plus faible variance ($\sigma = 4.80$), et meilleure survie (38.2/40 steps).

5.2 Off-policy vs On-policy

Le DQN (off-policy) réutilise les données du replay buffer, ce qui le rend plus sample-efficient en théorie mais moins stable (les données deviennent *stale*). Le PPO (on-policy) jette les données après chaque update mais bénéficie de données toujours fraîches. Sur highway-env avec des véhicules agressifs (environnement hautement stochastique), la stabilité de PPO est un avantage significatif.

5.3 SB3 vs implémentation manuelle

Notre DQN manual est compétitif avec les modèles SB3 (27.54 vs 27.14 pour DQN SB3) malgré l'absence d'environnements vectorisés et d'optimisations ingénierie. Il est aussi nettement plus rapide à entraîner (∼21 min pour 1 000 épisodes vs ∼24 min pour 100k steps SB3, grâce à l'absence d'overhead de logging et de vectorized env management).

5.4 Courbe d'entraînement du DQN manual

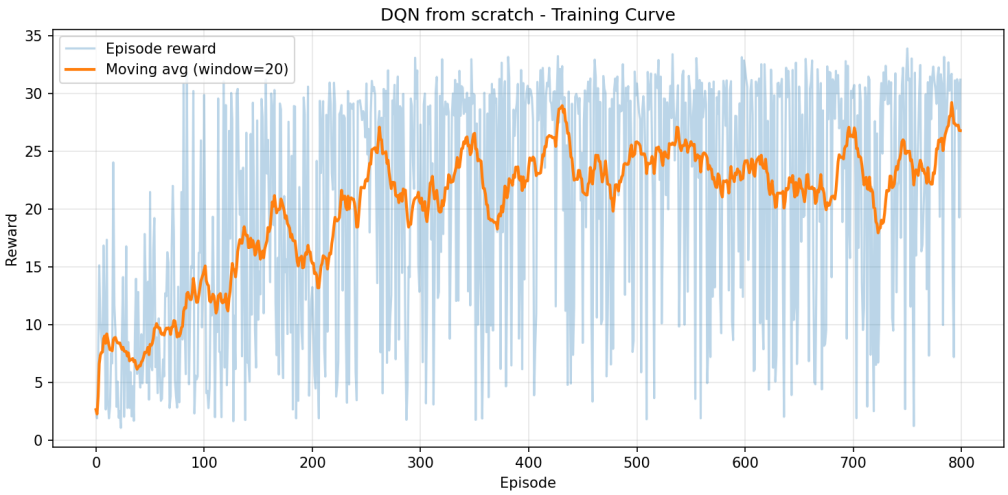


FIGURE 3 – Courbe d'entraînement du DQN manual : exploration forte au début (ε élevé), apprentissage rapide vers ép. 200–300, puis plateau en exploitation.

6 Overtraining et variance en RL

6.1 Le paradoxe de l'overtraining

Contrairement au supervised learning où plus de données signifie un meilleur modèle, en RL le modèle *génère ses propres données*. Une politique qui se dégrade produit des données de mauvaise qualité, créant un cercle vicieux. L'`EvalCallback` qui sauvegarde le best model est donc **indispensable** : le modèle final (dernier step) n'est souvent pas le meilleur.

Nos résultats le démontrent clairement : en Phase 2, les 3 algorithmes obtiennent des résultats inférieurs à la Phase 1 malgré un budget d'entraînement 3× supérieur. L'utilisation systématique de best checkpoints a été la clé pour récupérer les meilleures politiques.

6.2 Variance inter-runs

Le même algorithme avec les mêmes hyperparamètres peut donner des résultats très différents d'un run à l'autre, à cause de :

1. **Seeds aléatoires** : poids initiaux, ordre des transitions, épisodes rencontrés.
2. **Environnement stochastique** : véhicules à positions aléatoires, comportements variables.
3. **Composition du replay buffer** : différentes transitions mènent à des apprentissages différents.
4. **Échantillonnage de l'évaluation** : 15–30 épisodes restent un petit échantillon.

Idéalement, il faudrait reporter des résultats moyennés sur 3–5 seeds. Le temps limité ne nous l'a pas permis, mais nous reconnaissons cette limite.

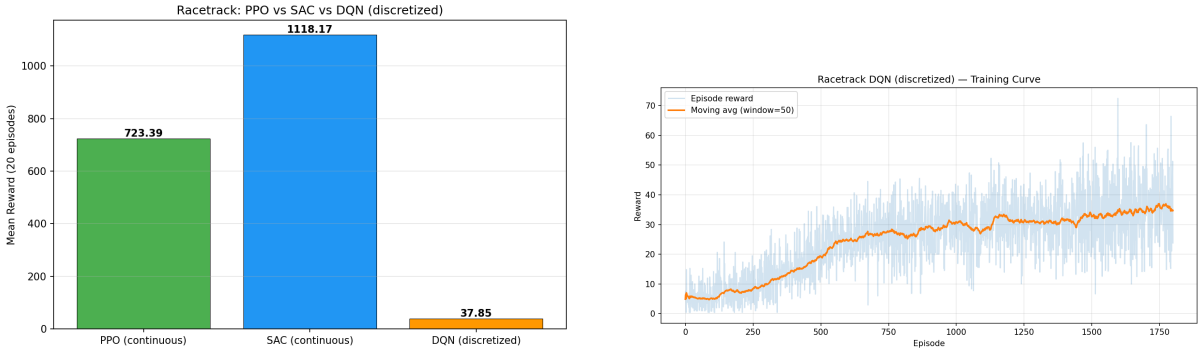
6.3 L'approche itérative comme clé du succès

Notre méthodologie **diagnostic** → **correction** → **retrain** a été la clé : chaque phase a identifié des problèmes spécifiques (Q-value overestimation, epsilon trop lent, overtraining) et les corrections ciblées ont systématiquement fonctionné. Le DQN Manual est passé de 8.35 à 27.54 (+230%) grâce à 5 corrections successives.

7 Bonus : Racetrack

En bonus, nous avons appliqué nos algorithmes à l'environnement `racetrack-v0`, un circuit avec des virages serrés nécessitant des actions continues (accélération, direction). Nous avons utilisé :

- **PPO (SB3)** : adapté naturellement aux actions continues,
- **SAC (SB3)** : Soft Actor-Critic, off-policy, spécialement conçu pour les espaces d'actions continus,
- **DQN manual** : avec discrétisation de l'espace d'actions.



(a) Benchmark racetrack.

(b) Courbe d'entraînement DQN.

FIGURE 4 – Résultats sur l'environnement racetrack (bonus).

Le SAC (off-policy, actions continues) s'avère particulièrement adapté à ce type d'environnement, tandis que le PPO reste compétitif.

8 Conclusion

Ce projet nous a permis de mettre en pratique les concepts fondamentaux du RL :

- **DQN** : comprendre le replay buffer, le target network, et le problème de Q-value overestimation.
- **PPO** : comprendre l'approche actor-critic, le clipping, et le trade-off on/off-policy.
- **Implémentation manuelle** : maîtriser chaque composant du DQN (réseau, buffer, epsilon-greedy, boucle d'entraînement).
- **Tuning** : l'importance cruciale des hyperparamètres (γ, lr) et des techniques de stabilisation (lr schedulers, gradient clipping, EvalCallback).

Le principal enseignement est que le RL est un domaine où l'**ingénierie expérimentale** compte autant que la théorie : diagnostiquer les problèmes, proposer des corrections ciblées, et valider itérativement est essentiel pour obtenir de bonnes performances.

Références

[1] V. Mnih et al., *Human-level control through deep reinforcement learning*, Nature, vol. 518, pp. 529–533, 2015.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, *Proximal Policy Optimization Algorithms*, arXiv :1707.06347, 2017.

[3] H. van Hasselt, A. Guez, D. Silver, *Deep Reinforcement Learning with Double Q-learning*, AAAI, 2016.

[4] E. Leurent, *An Environment for Autonomous Driving Decision-Making*, https://github.com/Farama-Foundation/HighwayEnv, 2018.

[5] A. Raffin et al., *Stable-Baselines3 : Reliable Reinforcement Learning Implementations*, JMLR, vol. 22, no. 268, pp. 1–8, 2021.

A Structure du code

Fichier	Description
`Final_Project.ipynb`	Notebook principal (entraînement, évaluation, figures)
`Final_Project.py`	Version Python du notebook (format percent)
`models/`	Poids des modèles entraînés (`.zip` SB3, `.pt` PyTorch)
`figures/`	Graphiques de benchmark et hyperparamètres
`videos/`	Vidéos de l'agent aléatoire et entraîné
`bonus/`	Environnement racetrack (notebook, modèles, figures)
`trainv2.py`, `trainv3.py`	Scripts d'entraînement (Phases 3 et 4)

Table 7 – Structure du dépôt.